

Patrones de Diseño: Expense Manager

Enero 2026

1. Introducción

Este documento detalla los patrones de diseño y principios de asignación de responsabilidades aplicados en el proyecto *Expense Manager*. La arquitectura se fundamenta en el patrón MVC, garantizando la integridad de los datos y una comunicación fluida entre la lógica de negocio y la interfaz de usuario.

2. Patrones Creacionales

Singleton (Instancia Única): Se implementa en las clases `Configuracion`, `ControladorSesion`, `FactoriaEstrategia` y `FactoriaImportadores`.

- **Configuracion:** Garantiza la unicidad del sistema al instanciar el `ControladorAppGastos` y los repositorios.
- **ControladorSesion:** Asegura un único punto de acceso global al usuario identificado.
- **Factorias:** Al ser *Singletons*, se centraliza la lógica de creación de objetos dinámicos en una única instancia global, optimizando el uso de memoria y evitando la creación redundante de objetos creadores.

Factory Method: Se utiliza para delegar la creación de objetos dinámicos de forma desacoplada:

- *FactoriaEstrategia:* Encargada de instanciar las estrategias de reparto para las cuentas compartidas (**RepartoEquitativo** o **RepartoPorcentual**).
- *FactoriaImportadores:* Centraliza la creación de componentes de importación según el formato del archivo.

3. Patrones Estructurales

Adapter: Este patrón se aplica tanto en la persistencia como en la importación. Permite que la aplicación trabaje con interfaces genéricas mientras que las clases concretas adaptan librerías externas (como Jackson para JSON o librerías de lectura de CSV) para cumplir con los contratos del sistema.

4. Patrones de Comportamiento

Observer (Modelo de Delegación de Eventos): El sistema utiliza un modelo de delegación de eventos donde el `ControladorAppGastos` notifica cambios de estado a los interesados. Las vistas se registran como oyentes para reaccionar ante eventos específicos (como la adición de un gasto), permitiendo que la UI se actualice de forma desacoplada.

Strategy: Se aplica en dos ámbitos diferenciados:

- *Reparto:* Permite intercambiar el algoritmo de cálculo de deudas en una cuenta compartida.
- *Alertas:* Se utiliza para definir la lógica de comprobación de límites (Mensual o Semanal). Cada estrategia encapsula el algoritmo que determina si un gasto debe disparar una alerta según el periodo temporal definido.

5. Arquitectura y Principios de Diseño

Además del patrón arquitectónico **MVC** (Modelo-Vista-Controlador), el diseño cumple con los patrones **GRASP**:

- **Experto:** Las responsabilidades se asignan a la clase que posee la información necesaria; por ejemplo, la cuenta compartida es la experta en calcular sus propios repartos entre miembros.
- **Creador:** La clase `Configuracion` y las Factorías asumen la creación de objetos complejos para reducir el acoplamiento y centralizar el control de instancias.
- **Controlador:** El `ControladorAppGastos` actúa como el primer objeto más allá de la capa de interfaz que recibe y coordina las operaciones del sistema, delegando el trabajo al modelo.